

Lab 9 Viva Study Guide

What CNN filters respond to: Top-8 images, receptive fields, Conv1 vs Conv2

Siddharth Narayan Mishra, 123CS0189

Contents

1	How to use this guide	2
2	Cheat sheet: numbers to know	2
3	Section 1: dataset, architecture, training setup	3
3.1	Architecture, line by line	3
3.2	Training configuration	3
3.3	Dataset choices (where I made assumptions)	4
4	Section 2: terminology	5
5	Sections 3 and 4: the maximally activating image	5
6	Experiment 1: training and verification	7
7	Experiment 2: Conv1 filter 2	7
8	Experiment 3: Conv1 versus Conv2	8
9	Experiment 4: selectivity across digit classes	9
10	Experiment 5: receptive-field mathematics	10
11	Experiment 6: PRE-ReLU versus POST-ReLU	11
12	Section 11: deliverables and results table	13
13	Trade-offs and design decisions	13
14	Things an examiner may push on (be honest)	14
15	Rapid-fire self-test	14

1 How to use this guide

Each section follows the assignment's own sections in order. Under each one there are the concepts, the numbers from my run, and questions an examiner can ask with a short answer. Questions marked **Trap** are places where a sensible-sounding answer is wrong.

The viva will be about *concepts and what the results mean*, so the guide spends most of its space there. Know the numbers in the cheat sheet cold. If you can derive a number ($28 \rightarrow 24$, 256 , 14×14), you do not have to memorise it.

The whole assignment in five sentences

1. Train a small LeNet on MNIST and check it is accurate (so its filters mean something).
2. For one filter, score every test image by the *largest* value that filter produces anywhere in its feature map, and keep the 8 images with the biggest scores.
3. For each of those images, find *where* the maximum was and which patch of the input the corresponding neuron can see (its receptive field).
4. Compare Conv1 (small RF, simple patterns) with Conv2 (large RF, more complex patterns), check whether filters are tied to digit classes, and verify the RF sizes by formula.
5. Check that using the raw (PRE-ReLU) values instead of the ReLU-ed (POST-ReLU) ones changes the picture only when responses are negative.

2 Cheat sheet: numbers to know

Quantity	Value and where it comes from
Conv1 output	$6 \times 24 \times 24$, since $28 - 5 + 1 = 24$ (no padding, stride 1)
After pool 1	$6 \times 12 \times 12$ (2×2 pool, stride 2 halves it)
Conv2 output	$16 \times 8 \times 8$, since $12 - 5 + 1 = 8$
After pool 2	$16 \times 4 \times 4 \Rightarrow \text{flatten} = 16 \cdot 4 \cdot 4 = 256$
FC sizes	$256 \rightarrow 120 \rightarrow 84 \rightarrow 10$
Conv1 params	$6(1 \cdot 25) + 6 = 156$
Conv2 params	$16(6 \cdot 25) + 16 = 2,416$
FC1, FC2, FC3 params	30,840, 10,164, 850
Total params	44,426 (FC1 alone is 69%; both conv layers together are about 6%)
Conv1 neurons	$6 \cdot 24 \cdot 24 = 3,456$
Conv2 neurons	$16 \cdot 8 \cdot 8 = 1,024$
RF: Conv1, S2, Conv2, S4	5×5 , 6×6 , 14×14 , 16×16
Jump after each	1, 2, 2, 4
RF area ratio	$196/25 = 7.84$
Iterations per epoch	$\lceil 50000/64 \rceil = 782$ (last batch has 16 images)
Background pixel after normalisation	$(0 - 0.1307)/0.3081 = -0.424$ (not 0)
Stroke pixel after normalisation	$(1 - 0.1307)/0.3081 = 2.82$
Test accuracy	98.55% on my 2,000 test images (1971/2000); 98.85% on all 10,000
Final train / val accuracy	99.30% / 98.75%

My chosen filters: Conv1 filter 2 (Exp 2), Conv2 filter 7 (Exp 3), Conv1 filters 1, 3, 5 and Conv2 filters 5, 10, 14 (Exp 4), Conv2 filter 3 (Exp 6). **None of these were assigned; I picked them.**

Be ready to say this.

3 Section 1: dataset, architecture, training setup

3.1 Architecture, line by line

Q. Where does 24×24 come from? Where does 256 come from?

Output size of a convolution is $\lfloor (n + 2p - k)/s \rfloor + 1$. With $n = 28$, $k = 5$, $p = 0$, $s = 1$ that is 24. Pooling 2×2 with stride 2 gives 12. Conv2: $12 - 5 + 1 = 8$, pooling gives 4. Conv2 has 16 filters, so the flattened vector is $16 \times 4 \times 4 = 256$.

Q. Why is the Conv1 map 24 and not 28? Original LeNet kept the size.

Original LeNet-5 took a 32×32 input (MNIST padded by 2 on each side). This assignment uses 28×28 with no padding, so every convolution shrinks the map by $k - 1 = 4$. Consequence for this lab: no receptive field ever goes outside the image (more in Experiment 5).

Q. Why does Conv2 have 6 input channels and 150 weights per filter?

A Conv2 filter is $6 \times 5 \times 5$: it looks at all 6 Conv1 feature maps at once. $6 \cdot 25 = 150$ weights plus 1 bias. A Conv1 filter has just $1 \cdot 25 = 25$ weights plus a bias, because the input has one channel.

Q. Why average pooling and not max pooling? Does it matter for this lab?

LeNet's original sub-sampling layers averaged. It matters for two reasons. (1) For the receptive-field formula nothing changes: both are $k = 2$, $s = 2$. (2) Max pooling commutes with ReLU ($\max(\text{relu}(z)) = \text{relu}(\max z)$), but average pooling does not, so the order Conv $\rightarrow$ ReLU $\rightarrow$ AvgPool is a real choice. Max pooling is more translation-invariant and keeps strong activations; average pooling smooths and is gentler on gradients.

Q. Why is there no softmax at the end?

CrossEntropyLoss in PyTorch already applies log-softmax internally and expects raw scores (logits). Adding softmax would apply it twice. FC3 outputs logits, which the assignment calls "final class evidence".

Q. Why ReLU?

It adds non-linearity (without it, stacked conv/linear layers collapse into one linear map), is cheap, and does not saturate for positive inputs, which helps SGD. Cost: a neuron whose input stays negative gets zero gradient (a "dead" ReLU).

Q. Where is the learning actually happening, in conv layers or FC layers?

Parameter-wise, mostly in the FC layers (FC1 has about 30.8k of 44.4k parameters). The conv layers have few parameters because of weight sharing: the same 25 weights are reused at all 576 positions. That is the point of convolution: far fewer parameters and translation equivariance.

3.2 Training configuration

Q. What does momentum 0.9 do with LR 0.01?

SGD with momentum keeps a running velocity $v \leftarrow 0.9v + g$, $w \leftarrow w - 0.01v$. In steady state the step is about $0.01/(1 - 0.9) = 0.1 \times$ the gradient, so it behaves like a 10 times larger learning rate but smoother, because noisy gradient directions partly cancel.

Q. Why batch size 64? What are the trade-offs?

Small batches give noisier gradients (acts as a regulariser, more updates per epoch: 782) but use hardware less efficiently. Large batches give smoother gradients and fewer updates per epoch, and often need a larger LR. 64 is a common middle value.

Q. What does “seed 42” guarantee and what does it not?

It makes initial weights and the shuffling order repeatable on the same machine and library version, so results can be reproduced. It does not make the result “the best”. A different seed would give slightly different filters, and so possibly different Top-8 images. Filters are only defined up to such accidents: filter 2 in my network need not look like filter 2 in yours.

Trap: Filter numbers are not meaningful across trained networks. “Filter 2” only has a meaning for a specific checkpoint. This is why the assignment insists on using one fixed checkpoint for Experiments 2 to 6.

Q. What is PyTorch default initialisation?

For Conv and Linear layers weights are drawn from $U(-1/\sqrt{\text{fan_in}}, 1/\sqrt{\text{fan_in}})$ (Kaiming uniform with $a = \sqrt{5}$) and biases from the same range. For Conv1 fan-in is 25, so the weights start in $[-0.2, 0.2]$. This keeps activation variance from exploding or vanishing across the layers.

Q. Why must test accuracy be at least 97% before continuing?

Filter visualisation is only informative if the filters were trained to be useful. An untrained or badly trained network still has a “Top-8” for each filter, but the patterns reflect random weights and not learned features. 97% is a sanity gate.

Q. What does `model.eval()` change here?

It switches off dropout and uses running statistics in batch norm. This network has neither, so numerically nothing changes here. It is still required as good practice, together with `torch.no_grad()` so no graph is built.

3.3 Dataset choices (where I made assumptions)

Q. You used 2,000 test images. The assignment also says 10,000. Which is it?

The assignment contradicts itself: Section 4 says rank all 10,000 test images, Experiment 2 says 2,000. I followed the Experiment 2 wording and used a fixed random subset of 2,000 test images (seed 42). The size is one constant in the code, so switching to 10,000 is a rerun. With more images the maximum scores can only go up or stay the same, and the Top-8 may change.

Q. Why a random subset and not the first 2,000 test images?

The first part of the MNIST test set is not guaranteed to be class-balanced or writer-balanced. A seeded random subset is representative and reproducible.

Q. You normalised the input. Does that matter for the filters?

Yes, and it is a good thing to understand. After normalisation the background is -0.424 , not 0, and strokes are $+2.82$. A kernel’s response to a blank patch is $b + (-0.424) \sum w$, which is generally not 0. This explains Conv1 filter 5 (see Experiment 4). The images and crops I display use the original 0 to 255 pixel values, not the normalised ones. The scores would be different (but similar in ranking) without normalisation.

Q. Why a validation split?

To watch generalisation during training without touching the test set. The test set is used once at the end. I did not use validation for early stopping or tuning.

4 Section 2: terminology

Know these definitions well enough to say the difference between neighbouring ones.

Term	Meaning	Example from this net
Filter / kernel	Learned weights (plus a bias)	Conv1 has 6, each $1 \times 5 \times 5$; Conv2 has 16, each $6 \times 5 \times 5$
Feature map	Output of one filter over all positions	Conv1 filter 2 gives a 24×24 map
Neuron	One element $A[c, y, x]$ of a feature map	Conv2 neuron ($c=7, y=7, x=7$)
Activation	Numerical value of a neuron	11.93
PRE-ReLU Z	Raw convolution output + bias	can be negative
POST-ReLU A	$\max(0, Z)$	never negative
Receptive field	Input region that can influence the neuron	5×5 (Conv1), 14×14 (Conv2)
Classification output	Final logits from FC3	10 numbers

Q. Is a filter the same as a neuron?

No. A filter is a set of weights; a neuron is one output value produced when that filter is applied at one location of one specific image. One filter corresponds to 576 neurons in Conv1 (24×24) and 64 in Conv2 (8×8) for each input image. Because the weights are shared, all of these neurons are the same detector applied at different positions.

Q. What does “the receptive field is not a causal attribution map” mean?

The RF says which pixels *could* influence a neuron, as determined by the architecture. It does not say which pixels *did* influence it, or by how much. Some pixels in the box may have near-zero weight, or the active input may only be a few pixels. To claim a pixel caused a response you would need to change it and see the output change (occlusion or gradient methods).

Q. What is the difference between theoretical and effective receptive field?

Theoretical: the whole region that is connected, which is what we compute. Effective: the region that has noticeable influence, which is usually smaller, concentrated around the centre and roughly Gaussian, because central pixels connect to the neuron through many more paths. This lab only uses the theoretical one.

5 Sections 3 and 4: the maximally activating image

$$S_i = \max_{y,x} A_i(c, y, x), \quad (y_i^*, x_i^*) = \arg \max_{y,x} A_i(c, y, x).$$

Q. Explain the score in words.

For a fixed filter c and an image i , take the whole 24×24 (or 8×8) POST-ReLU map and keep only its biggest value. That value says “how strongly does this image contain the filter’s pattern

somewhere". The location of that value says where. Sort the images by this score and take the first 8.

Q. Why the maximum over positions and not the mean or sum?

A filter looks for a small local pattern. In a typical digit it matches in only a few places and is near zero elsewhere. A mean would be diluted by the large empty background and would favour images where the filter fires weakly in many places. The max asks "is the pattern present anywhere" and so is position-independent. The trade-off is that one noisy location can decide the score, and the score ignores how often the pattern occurs.

Q. What does "filter-level, not neuron-level" mean?

We fix the filter and let the location vary per image (the maximum can sit anywhere). A neuron-level experiment would fix also the location (y, x) and rank images by that single neuron's value. In my Figure 1 the max location differs in every image, which is evidence that this is filter-level.

Q. Why does the location differ across images?

Convolution is translation-equivariant: the same weights are applied everywhere, so if the pattern appears at a different place the response appears at a different place. Different images contain the pattern at different positions.

Edge cases of the definition

- **Ties.** If two images have the same score the order is arbitrary. This is rare for continuous scores but certain for images whose score is exactly 0 after ReLU (Experiment 6). Inside one image, ties between locations are broken by taking the first one in row-major order.
- **Score equals zero.** If the whole POST-ReLU map is 0 then the "location of the maximum" is meaningless (any location is a maximum). I never have this in my Top-8, but you should know it cannot be interpreted.
- **Bias towards ink.** Images with a lot of bright stroke produce large responses for filters with positive weights. Conv1 filter 1 has large positive weights in the middle of its kernel (the sum of weights is +2.32), so its Top-8 are patches that are filled with ink. High score does not imply a specific shape.
- **Top-8 shows the extreme, not the typical.** Eight images out of 2,000 are a very small sample from one end. They show what makes a filter fire hardest, not what it does on a typical image.
- **Sample dependence.** The Top-8 depends on which 2,000 images were scanned. For Conv1 filter 2 the eight scores are within 11.79 to 11.97, so a small change to the image set can reorder or replace images.
- **No causal claim.** A visual pattern shared between Top-8 patches is a description of correlation. It is a hypothesis about what the filter detects.

Q. How is this different from activation maximisation (gradient ascent on the input)?

Gradient ascent starts from noise and optimises the pixels to maximise a filter, producing a synthetic image that may not look natural. Dataset search (this lab) takes real images, so patterns are realistic and the result reflects the data distribution, but it can only show what appears in the dataset, and correlated features (e.g. "2s always have a bottom stroke") are confounded. Related but different methods: saliency maps and Grad-CAM (which pixels matter for a particular output) and occlusion (remove a patch and see the change).

6 Experiment 1: training and verification

Result. Training loss goes $0.481 \rightarrow 0.021$ over 10 epochs, train accuracy $84.8\% \rightarrow 99.3\%$, validation $95.9\% \rightarrow 98.75\%$. Test accuracy 98.55% (2,000 images), 98.85% (all 10,000).

Q. Why is epoch-1 train accuracy (84.8%) so much lower than validation accuracy (95.9%)?

Training accuracy is a running average over the epoch, computed while the weights are still improving, so the early batches count with a bad network. Validation accuracy is measured after the epoch with the improved weights. Also there is no dropout or augmentation, so it is not a regularisation effect. By epoch 10 the numbers are close to a proper end-of-epoch value.

Q. Is the model overfitting?

A little. Train accuracy (99.30%) is above validation (98.75%), a gap of 0.55 points, and validation accuracy plateaus at around 98.4 to 98.75% from epoch 4 while training loss still falls. That is mild overfitting, not a problem for this lab. Validation also dips at epoch 8 (98.40%), which is normal fluctuation with a constant learning rate.

Q. 98.55% on my test subset but 98.85% on the full set. Which is right?

Both are true for their sets. The standard error of accuracy on 2,000 images is about $\sqrt{0.0145 \cdot 0.9855 / 2000} \approx 0.27$ points, so a 0.3 point difference is within noise. The full 10,000 is the more precise estimate of the model's accuracy.

Q. Why do we also record shapes?

It is a check that the implementation matches the specified architecture. The shapes ($6 \times 24 \times 24$ etc.) are the ones in the brief.

7 Experiment 2: Conv1 filter 2

Result. Top-8 digits 7,7,7,2,5,7,3,7 (7 appears 5 times). Scores 11.97 down to 11.79. Max locations (5, 12), (6, 19), (5, 5), (1, 9), (4, 14), (6, 10), (19, 9), (5, 17). Each RF is 5×5 and inside the image.

What the filter is. The actual kernel of Conv1 filter 2 has negative weights in its top two rows (about -0.1 to -0.55) and positive weights in its lower rows (up to $+0.56$). Normalised background is negative, so negative weights over background contribute positively, and positive weights over ink contribute positively. The best match is therefore: *dark in the upper rows, ink in the lower rows*, i.e. the upper edge of a horizontal-ish stroke. That is what the crops show, and it is why 7s (with a top bar) dominate.

Q. What do the Top-8 patches have in common?

Each 5×5 patch has background in its upper rows and the top edge of a stroke in its lower rows. In the 7s this is the top bar, in the 2 and 5 the top of the digit, in the 3 the lower curve.

Q. Does this mean filter 2 is a “7 detector”?

No. Five of eight are 7s, but a 2, a 5 and a 3 are also in the list, and the patch is a 5×5 piece of stroke. The filter detects a local edge. 7s rank high because they have a long clean top bar, not because the filter knows the digit 7. A 5×5 field cannot even see a whole digit (digits are about 20 pixels tall).

Q. The RF crop of the rank-7 image (the “3”) comes from the bottom of the digit, not the top. Is that a mistake?

No. The score is a maximum over *all* positions. In that image the strongest response happened to be at the lower curve, which also has background above and a stroke below. The location is chosen per image.

Q. Scores are in a narrow band (11.79 to 11.97). What does that tell you?

Many images contain a near-perfect match of the pattern (a clean bar over a black background), and the response saturates near the value of an ideal match. The exact ranking inside the Top-8 is therefore not robust. Any small change (different images, different seed) could reorder them.

Q. Why does the maximum location move between images, yet the RF is always 5×5?

The RF size depends only on the architecture (kernel sizes and strides). Its position depends on where the maximal neuron is: neuron (y, x) in Conv1 sees input rows $y..y+4$ and columns $x..x+4$.

8 Experiment 3: Conv1 versus Conv2

Result for Conv2 filter 7. Top-8 digits 2,2,2,8,2,2,2,2 (2 appears 7 times). Scores 11.93 to 11.66. Max locations are all in the last row of the 8×8 map: (7, 7), (7, 7), (7, 6), (7, 5), (7, 7), (7, 7), (7, 6), (7, 6). RF 14×14, rows 14 to 27, columns between 10 and 27.

Q. Compare Conv1 and Conv2 on RF size, activated regions, pattern complexity and digit distribution.

RF: 5×5 against 14×14 (about 7.8 times the area). **Regions:** Conv1 maxima are spread over the image (rows 1 to 19); Conv2 filter 7 maxima are all in one corner (bottom right). **Complexity:** a Conv1 crop is one edge, a Conv2 crop has several strokes and their arrangement (a stroke ending and the empty space around it). **Digits:** Conv1 filter 2: 7 five times; Conv2 filter 7: 2 seven times. This is more concentrated, but it cannot be generalised, because Conv2 filters 10 and 14 are not concentrated.

Q. What evidence supports a change in representation from Conv1 to Conv2?

(1) The receptive field grows from 5×5 to 14×14, so Conv2 can respond to arrangements that do not fit in 5×5. (2) The Conv1 crops are single edges or strokes that appear in many digits; the Conv2 crops contain bends, endings or two strokes together. (3) Some Conv2 filters (5 and 7) are tied to one digit much more than any Conv1 filter I looked at. (4) Structurally, Conv2 takes Conv1 feature maps as input, so it combines edges rather than pixels. I should add the caveat that this rests on a few filters and a subset of the test data.

Q. Should Conv2 filters represent whole digits?

No, and the assignment says not to assume it. A 14×14 field covers only a quarter of the image area (196 of 784 pixels). A whole digit needs the full image. Conv2 gives parts (curves, junctions, stroke endings). The combination into digit decisions is the job of the FC layers, whose neurons see all 256 inputs.

Q. Conv2 filter 7 always peaks at the bottom-right. Why? Is it a bug?

Not a bug. Three facts together explain it. (1) Neuron (7, 7) is the last one, and its RF is rows 14 to 27, columns 14 to 27. (2) MNIST digits are size-normalised into a 20×20 box centred in the 28×28 image, so the outer 4-pixel border is empty, and a RF touching the image border includes a strip of guaranteed empty space. (3) The filter seems to encode “a stroke ends here and there is empty space to the right and below”, so the ideal match is exactly at the edge of the image. The

trained weights can use the empty border as part of the pattern, so filters learn about absence as well as presence. This also means the filter is position-sensitive in a way Conv1 filters are not.

Q. Positions (7,5), (7,6), (7,7) are neighbours in the map. How far apart are their RFs?

Two input pixels per step (jump = 2). Columns 10 to 23, 12 to 25, 14 to 27. The boxes overlap heavily, because 14 is much larger than the step of 2.

Q. Can I compare an activation of 11.9 in Conv2 with 11.9 in Conv1?

No. Activation values depend on the weights, biases and scale of each filter. They are only comparable inside one filter (to rank images). Conv1 filter 5 peaks at 2.7, and Conv2 filter 14 at 26.1, which says nothing about which is more “important”.

9 Experiment 4: selectivity across digit classes

The rules (apply to the Top-8 digit labels).

- *Class-associated*: at least 6 of 8 share a digit.
- *Multi-class*: at least 3 digit classes appear, and no digit has 6 or more.
- *Shared visual pattern*: the RF patches show a common pattern even across different digits. This is judged by looking, not counted.

Filter	Top-8 distribution	Count rule	What the patches show
Conv1 f1	1:3, 4:2, 7, 8, 9	multi-class	thick ink filling the patch, dark column right and row bottom
Conv1 f3	0:4, 7:2, 9:2	multi-class	mostly ink, dark on left edge and lower right
Conv1 f5	6:3, 5:2, 3, 4, 9	multi-class	almost empty patch (empty-space detector)
Conv2 f5	4:6, 0:2	class-associated	two vertical strokes with a gap
Conv2 f10	0:5, 2:2, 4:1	multi-class	curved left stroke into a bottom bar
Conv2 f14	3:2, 4:2, 0,2,5,6	multi-class	diagonal stroke to lower right

What the kernels say (check yourself). Conv1 filter 1 has large positive weights in the middle and negative ones at the right column and bottom row, which is exactly the “ink in the middle, dark right and bottom” that the crops show. Conv1 filter 3 is positive in the centre-left, negative at the right and lower right. **Conv1 filter 5 has all 25 weights negative** (sum -5.33) and bias $+0.33$. Its response to a completely blank patch is $0.33 + (-0.424)(-5.33) \approx 2.59$, which is almost the same as its best score (2.72). So this filter is strongest where there is *no* ink, every image contains a blank patch, and every image scores at least 2.59. Its Top-8 is separated from the rest by only about 0.1 and is basically noise (the faint grey pixel in each crop sits at the one position with a positive weight, $+0.05$).

Q. How did you classify Conv1 filter 1? It has 3 ones and 2 fours.

By the count rule: five distinct classes, largest count 3, so multi-class. Looking at the patches, the same bright blob appears for 1, 4, 7, 8 and 9, so it is also a shared visual pattern. The two labels are not exclusive: one is a statistic on labels, the other a visual judgement.

Q. Conv2 filter 10 has 0 in five of eight. Why is it “multi-class” and not class-associated?

Because the rule needs at least 6. Five of eight is just below the threshold. Honest answer: the threshold is a convention. 5 of 8 is far from chance (if the Top-8 were drawn at random from 10

balanced classes, the chance of any single class getting 5 of 8 is about $\binom{8}{5}0.1^50.9^3 \approx 4 \times 10^{-4}$, so the filter is clearly biased towards 0s. I would say “multi-class by the rule, leaning to 0, with a pattern (curved stroke with a bottom bar) that 0, 2 and 4 share”.

Q. Why is Conv2 filter 5 class-associated, and what does it detect?

Six of eight are 4s (probability of this by chance about 2×10^{-4} summed over classes), and the other two are 0s. The crops show two roughly vertical strokes separated by a gap. A 4 (open top) has two vertical strokes there, and a 0 shows its left and right sides in the same arrangement. So the filter detects two parallel vertical strokes, and 4 is the digit that provides them most strongly. It is better described as a stroke-arrangement detector that correlates with 4.

Q. Does “class-associated” mean the filter is a classifier for that digit?

No. It only means that the images which excite it most are mostly of that class. A filter can be class-associated and still fire on many other digits at lower strength. We looked at only the top 8 out of 2,000, and to claim true selectivity you would compute its mean activation per class (or an AUC for “digit d vs the rest”) over all images.

Q. Why can several Conv1 filters be multi-class while one Conv2 filter is class-associated?

Conv1 sees 5×5 pieces: edges and blobs, which every digit has. Conv2 sees 14×14 and can see a combination, such as two parallel strokes, that is typical for a particular digit. This is the expected direction: deeper layers are more class-specific. But Conv2 filters 10 and 14 are multi-class, so the effect is a tendency and not a rule.

Q. What are the limits of this experiment?

Only 8 images per filter (very small sample); only 2,000 test images scanned; only 3+3 filters; thresholds (6 of 8, 3 classes) are arbitrary; the shared-pattern judgement is subjective; and the labels describe the whole image while the patch is only a part of it (a “4” image may fire the filter on a stroke that is not specific to 4).

10 Experiment 5: receptive-field mathematics

Recurrence. Start with $r_0 = 1$, $j_0 = 1$. For a layer with kernel k and stride s :

$$r_l = r_{l-1} + (k - 1)j_{l-1}, \quad j_l = j_{l-1}s_l.$$

Here j (the *jump*) is the distance in input pixels between two adjacent neurons of that layer.

Layer	k	s	r_l	j_l
Conv1	5	1	$1 + 4 \cdot 1 = 5$	1
S2 (pool)	2	2	$5 + 1 \cdot 1 = 6$	2
Conv2	5	1	$6 + 4 \cdot 2 = 14$	2
S4 (pool)	2	2	$14 + 1 \cdot 2 = 16$	4

Q. Explain the formula intuitively. Why $(k - 1) \cdot j_{l-1}$?

A kernel of size k covers k neurons of the previous layer. Neighbouring neurons of the previous layer are j_{l-1} input pixels apart. So the k neurons span $(k - 1)j_{l-1}$ extra pixels beyond the field of the first one, which already has size r_{l-1} .

Q. Why does the jump double after pooling and not after a convolution?

Jump multiplies by the stride. Convolutions here have stride 1 (jump unchanged); pooling 2×2 has stride 2 (jump doubles). After S2, neighbouring neurons are 2 input pixels apart; after S4, 4 pixels.

Q. Derive the Conv2 RF directly, without the formula.

One Conv2 neuron looks at a 5×5 block of pooled maps. Each pooled neuron is the average of a 2×2 block of Conv1 neurons, so a 5×5 block of pooled neurons covers a 10×10 block of Conv1 neurons. Each Conv1 neuron sees 5×5 pixels, and adjacent ones overlap, so a 10-wide block of Conv1 neurons covers $10 + 5 - 1 = 14$ pixels.

Q. Where exactly is the RF of Conv2 neuron (y, x) ?

Rows $2y$ to $2y + 13$ and columns $2x$ to $2x + 13$ (offset 2 per step because the jump is 2). Check: $(7, 7)$ gives rows 14 to 27, the last valid row.

Q. Why does the check “after S2 RF = 6×6 ” matter if we only visualise Conv layers?

It checks that the recurrence is being used properly, and pooling does change the field seen by Conv2. Only Conv1 and Conv2 neurons are visualised, so 5×5 and 14×14 are what appear in the figures.

Q. What happens when the RF extends beyond the image? Does it happen in your network?

It happens when the layers use padding: the border neurons then see padded zeros outside the image. The assignment asks to report the theoretical size and the visible intersection with the 28×28 image. Example: with padding 2 in Conv1, neuron $(0, 0)$ has a theoretical RF of 5×5 covering rows $-2..2$, columns $-2..2$, and the visible part is 3×3 . **In this network there is no padding, so it never happens:** the largest Conv1 neuron $(23, 23)$ sees pixels 23 to 27 and the largest Conv2 neuron $(7, 7)$ sees 14 to 27, so theoretical equals visible everywhere. The check is that the box in the figure is the same size as the theoretical size.

Q. What would change the RF? (padding, stride, dilation, depth)

Larger kernels, more layers, larger strides and dilation all increase it. Padding does not change its size, only whether parts of it fall outside the image. Pooling increases the jump, which makes later layers' RF grow faster. So a deeper network sees more context, but with a theoretical field larger than the image, the extra part is only padding.

Q. What is the RF of an FC1 neuron?

The whole image. The 4×4 grid of S4 outputs has jump 4 and RF 16 each, so together they span $16 + 3 \cdot 4 = 28$ pixels, the full width. This is why whole-digit evidence is formed in the FC layers.

Q. Why is the RF not enough to say what the neuron “looks at”?

Because it is only the region of possible influence. A 14×14 field with a filter that responds to a thin diagonal stroke uses only the pixels along that stroke. The red box is an upper bound on the relevant area.

11 Experiment 6: PRE-ReLU versus POST-ReLU

Result (Conv2 filter 3). Top-8 images, their values and locations are identical in the two modes: indices 506, 1149, 535, 983, 1342, 1194, 302, 1499, scores 18.07 down to 17.58. Six of eight maxima

are in row 0 of the 8×8 map. For **20 of 2,000 images** the PRE-ReLU maximum is negative. The eight lowest scores (-0.57 to -0.27) are all images of digit 1.

Q. Why are the Top-8 identical?

Since ReLU is monotone non-decreasing, $\max_{y,x} \max(0, z) = \max(0, \max_{y,x} z)$. So $S^{\text{post}} = \max(0, S^{\text{pre}})$. When $S^{\text{pre}} > 0$, the two are equal, and the argmax location is also the same. The rankings differ only for images whose $S^{\text{pre}} \leq 0$. The Top-8 have large positive values, so the two modes cannot differ there. This was predictable before running it, which you should say.

Q. When do PRE and POST give different answers?

At the bottom of the ranking, for images where every response is negative. POST sets the score to 0 for all of them, so they tie and their ordering is lost. PRE still orders them. Their argmax locations are also no longer meaningful in POST (all zeros), while PRE gives the least negative place.

Q. If all PRE values are negative, what is the maximum and how should I describe it?

The maximum is still negative: it is the *least negative* value. It must not be called an activation or “the filter fires here”. It means that the filter’s pattern is least absent at that location. After ReLU it is exactly 0.

Q. Why are all eight lowest images 1s?

The result is that 1s are the digit that excites this Conv2 filter least. A plausible reason is that 1s are thin and have little ink, so nearly all positions look like background to filter 3, and the filter’s pattern needs something that a 1 does not have. I do not have a proof from the weights, so I would say “consistent with”, not “because”.

Q. Why is POST-ReLU convenient for interpretation?

It keeps only positive evidence, so “0” reads as “pattern not present” and a larger number reads as “more present”. PRE values mix two things: positive (evidence for) and negative (evidence against), and large negative numbers are easy to misread as strong activity if you only look at magnitude. POST-ReLU is also what the next layer actually receives. The cost: you lose information on *how* absent the pattern was, and many different PRE values collapse to 0.

Q. What is a dead ReLU and how is it related?

A unit whose pre-activation is negative for every input outputs 0 always and receives zero gradient, so it stops learning. In our scan no Conv1 filter and only Conv2 filters 1, 3 and 16 (4, 20 and 4 images respectively) ever have an all-negative image, so there are no dead filters here, just filters with rare all-negative images.

Q. Why did you choose Conv2 filter 3 for this experiment?

Honest answer: for most filters, every image has a positive maximum, which would make the comparison trivial (identical rankings and no negative cases). Filter 3 has 20 images with a negative maximum, so the difference between the two modes can actually be seen. It was my own choice since none was assigned.

12 Section 11: deliverables and results table

Q. Why exactly three figures?

The assignment fixes them: Conv1 Top-8, Conv2 Top-8 with RFs, and a selectivity comparison. Each shows the max-activation location and the RF region (red box and a cyan marker at its centre).

Q. In your figures, what do the red box and the cyan plus mean?

Red box: the theoretical RF of the maximum-response neuron in input coordinates. Cyan plus: the centre of that box, a marker of the maximum-activation location mapped to the input. Titles give digit, score and the feature-map location (y, x) , which is *not* a pixel coordinate.

Trap: The location (y, x) in the table is in feature-map coordinates (0 to 23 for Conv1, 0 to 7 for Conv2), not in image pixels. For Conv2 multiply by 2 to get the top-left pixel of the RF.

Q. Summarise the main table in two lines.

Conv1 filters have a 5×5 RF, mixed digit distributions, and respond to local stroke or edge patterns. Conv2 filters have a 14×14 RF, can be tied to one digit (filter 5: 4s, filter 7: 2s) and respond to arrangements of strokes, but are not whole-digit detectors.

13 Trade-offs and design decisions

- **2,000 vs 10,000 test images.** More images give a more reliable Top-8 and a better estimate of accuracy; fewer images are quicker and what Experiment 2 literally states. Either way, the Top-8 is a small, extreme sample.
- **Max vs mean score.** Max: position-independent, sensitive to a single strong response, ignores frequency. Mean: stable but diluted by background.
- **Average vs max pooling.** Average keeps information about all values in the window and is smoother; max keeps the strongest response and commutes with ReLU. Neither changes the RF formula.
- **Small RF vs large RF.** Large RF sees more context but less spatial detail, and costs more layers or larger kernels. Conv1 gives precise localisation of simple patterns, Conv2 gives context.
- **Real images vs synthesised images** (dataset search vs gradient ascent). Real images are interpretable and in-distribution but limited and confounded; synthesised images isolate what the filter wants but can look unnatural.
- **PRE vs POST.** POST is clean to interpret but loses magnitude of negative evidence; PRE is complete but harder to read.
- **Normalising the input.** Helps training, but shifts the meaning of “blank” (background $\neq 0$) and so changes what a kernel’s bias and weights mean. This is behind the odd Conv1 filter 5.
- **No padding.** Smaller feature maps and no RF overflow, but the border pixels are used by fewer neurons than the centre pixels.

14 Things an examiner may push on (be honest)

Q. The assignment says instructor-assigned filters. Which did you use?

No filters or checkpoint were given to me. I trained my own model per the specification and chose filters myself (listed in Section 2), picking one Conv2 filter in Experiment 6 so that negative maxima exist. If you give me the assigned numbers I change one constant and rerun the six scripts, and the results (digits, scores, patterns) will differ because those are filter-specific.

Q. Your results would differ for a different seed. Does that make them wrong?

No. Different training runs give different filters, so “filter 2” is not a stable object. What should be stable are the qualitative statements: Conv1 filters are local edge/stroke/blob detectors, the RF sizes are 5 and 14, Conv2 filters are more complex and sometimes digit-biased. Those are what the experiments are meant to show.

Q. What would you do to make the digit-selectivity claim stronger?

Use all 10,000 images and compute per-class mean activation (or AUC) for each filter, instead of looking at the Top-8 only. Use occlusion or gradient attribution to see which pixels matter inside the RF. Repeat over several seeds.

Q. Which statement in your report is the weakest?

The shared-pattern descriptions (Experiment 4), because they are my visual reading of eight crops. I checked the Conv1 ones against the kernel weights and they agree (filters 1, 2, 3, 5). The Conv2 descriptions I could not check against weights directly, since a Conv2 kernel is 6 channels of 5×5 and not directly readable in pixel space.

15 Rapid-fire self-test

Cover the right-hand side and answer aloud.

Question	Answer
Conv1 output shape?	$6 \times 24 \times 24$
Why 256?	$16 \times 4 \times 4$
Total parameters?	44,426
Params in Conv1?	156
Which layer has most parameters?	FC1 (30,840)
RF after Conv1 / S2 / Conv2 / S4?	5 / 6 / 14 / 16
Jump after S4?	4
RF of an FC1 neuron?	the full 28×28 image ($16 + 3 \cdot 4$)
Conv2 neuron (y, x) covers?	rows and columns $2y..2y + 13$, $2x..2x + 13$
Definition of S_i ?	max over positions of POST-ReLU map of the filter
Is the max location the same for all images?	No, filter-level not neuron-level
Relation between S^{post} and S^{pre} ?	$S^{\text{post}} = \max(0, S^{\text{pre}})$
When do PRE and POST Top-8 differ?	only if some Top-8 scores are ≤ 0 or in the tied zero region
Class-associated threshold?	≥ 6 of 8 same digit
Multi-class condition?	≥ 3 digits and none with ≥ 6
Only class-associated filter in my results?	Conv2 filter 5 (4:6)
What is odd about Conv1 filter 5?	all-negative kernel: empty-space detector, scores almost equal to the blank-patch response 2.59
Why do Conv2 filter 7 maxima sit at the corner?	last map row/column, RF touches the border, filter uses empty border as part of its pattern
Does the RF show causation?	No, only possible influence
Why filters must come from one fixed checkpoint?	filter indices have meaning only in a given trained network
Test accuracy?	98.55% (2,000), 98.85% (10,000)
Background value after normalisation?	-0.424